

DESARROLLO DEL BACKEND DE UNA PLATAFORMA E-LEARNING CON SOPORTE
SCORM (DOMINIO ESTUDIO)

Omar Esneyder Cruz Parada

C.C. 10933854482

Informe final de Trabajo de Grado como Requisito Para Optar al Título de Tecnólogo en
Gestión de redes y sistemas Tele-informáticos

Asesor de Trabajo de Grado:

Ing. Elkin E. Rojas L.

Instituto Superior de Educación Rural (ISER)

Facultad De Ingenierías e Informática

Tecnología en Gestión de redes y sistemas Teleinformáticos

Pamplona, Norte de Santander, Colombia

2025

Dedicatoria

Dedico este trabajo a mi familia, que ha sido mi mayor fuente de fortaleza, inspiración y acompañamiento a lo largo de mi vida.

A mis padres, quienes, con su amor incondicional, sus sacrificios silenciosos y su ejemplo diario me enseñaron a luchar por mis sueños, a mantenerme firme ante las dificultades y a creer en el poder del esfuerzo constante. Gracias por cada palabra de ánimo, por cada gesto de apoyo y por cada momento en el que, aun sin decirlo, hicieron sentir que nunca caminaba solo.

A quienes han confiado en mí incluso cuando yo mismo dudaba, les agradezco profundamente por recordarme mi capacidad y por motivarme a seguir avanzando. Cada logro alcanzado es también resultado de su presencia, de su paciencia y de la fe que han depositado en mí.

A mi familia extendida, amigos y personas cercanas que siempre estuvieron allí con una palabra de apoyo, un consejo oportuno o simplemente con su compañía, les dedico también este logro, porque de una u otra manera han formado parte del camino que hoy se ve reflejado en este trabajo.

Este proyecto es más que un requisito académico; es la demostración del crecimiento personal, del aprendizaje continuo y del esfuerzo compartido con quienes han marcado positivamente mi vida. A todos ustedes, gracias por ser mi sostén, mi impulso y mi razón para continuar.

Agradecimientos

En este importante cierre de mi proceso académico, quiero expresar mi más profundo agradecimiento a todas las personas que hicieron posible la culminación de esta etapa. Este logro no es solo mío, sino también de quienes me acompañaron con su apoyo, confianza y cariño durante todo el camino.

A mi **madre**, quien ha sido mi mayor fortaleza y ejemplo de vida. Gracias por tu amor incondicional, tus consejos, tus oraciones y por estar siempre presente en cada paso que di. Tu apoyo constante fue el motor que me impulsó a seguir adelante incluso en los momentos más difíciles.

A mi **familia**, por su paciencia, comprensión y aliento en cada jornada de esfuerzo. Gracias por creer en mí y por motivarme a alcanzar mis metas, aun cuando las circunstancias no eran fáciles.

A los **profesionales y docentes** que me brindaron su conocimiento y guía, especialmente a mi **asesor académico**, por sus orientaciones, enseñanzas y disposición para acompañarme durante el desarrollo de este proyecto. Su apoyo fue clave para fortalecer mis habilidades y alcanzar los objetivos propuestos.

A mis **compañeros de estudio**, con quienes compartí experiencias, aprendizajes, retos y momentos inolvidables. Gracias por su amistad, colaboración y por ser parte fundamental de este proceso de crecimiento personal y profesional.

Finalmente, a todas las personas que, directa o indirectamente, contribuyeron a que este trabajo se hiciera realidad, les expreso mi más sincero agradecimiento. Sin cada uno de ustedes, este logro no tendría el mismo valor.

Tabla de contenido

Resumen	XIII
Abstract	XIV
Introducción	XV
Planteamiento del Problema	XVII
Justificación	XVIII
Objetivos	XIX
Objetivo General	XIX
Objetivos Específicos	XIX
Marco Conceptual	XXIV
Marco Histórico	XXVII
Metodología	XXIX
Enfoque general de la metodología ágil–Scrum	XXIX
Aplicación de la metodología en el proyecto	XXX
Justificación	XXXIII
Resultados del Plan de Actividades	XXXVII
Primer Objetivo	XXXVII
Primera actividad	XXXVII
Segunda actividad	XLI
Tercera actividad	XLII
Segundo Objetivo	XLV
Primera actividad	XLV
Segunda actividad	XLVIII
Tercera actividad	XLIX
Tercer Objetivo	LI
Primera actividad	LI
Segunda actividad	LV
Tercera actividad	LVI
CRUD de actividades	LVI
Cuarto Objetivo	LIX

Primera actividad.....	LIX
Segunda actividad	LX
Tercera actividad.....	LXI
Conclusiones	LXIII
Recomendaciones	LXV

Tabla de figuras

Fig. 1. Línea de Código, Muestra de versión Laravel.....	XXXVII
Fig. 2. Línea de Código, Interacción de la base de datos con Docker.	XXXVIII
Fig. 3. Docker, Contenedores Docker.	XXXVIII
Fig. 4. Estructura, Diagrama de Arquitectura del Backend Modular.	XXXIX
Fig. 5. Línea de Código, Autenticación de Roles.	XL
Fig. 6. Uso de módulo SCORM.....	XLI
Fig. 7. Uso de módulo SCORM.....	XLII
Fig. 8. Modelo UML Modelo Entidad–relación de la Base de Datos.....	XLIII
Fig. 9. . Línea de Código, Autenticación de Tokens.	XLIV
Fig. 10. Línea de Código, Inicio de Sesión.....	XLV
Fig. 11. Línea de Código, Registro de Usuarios.	XLVI
Fig. 12. Línea de Código, Revocación de tokens.	XLVII
Fig. 13. Protección de Rutas con Middleware.	XLVII
Fig. 14. Línea de Código, Creación de un Seeder para Admin, Cliente, User.....	XLVIII
Fig. 15. Línea de Código, Control de acceso API-middleware.	XLIX
Fig. 16. Línea de Código. Creación/Edición de cursos.....	LI
Fig. 17. Línea de Código, Eliminación y CRUD de Cursos.	LII
Fig. 18. Línea de Código, Filtros avanzados (Cursos).....	LIII
Fig. 19. Interfaz Laravel, Gestión de Archivos Multimedia.	LIV
Fig. 20. Línea de Código, Comunicación de Endpoints con Vue3.	LIV
Fig. 21. Documento, Endpoints para consumo Frontend (Autenticación).	LV
Fig. 22. Interfaz, Panel Administrativo en Filament de Actividades.	LVI
Fig. 23. Línea de Código, Modelo: Cursos->Módulos.	LVII
Fig. 24. Línea de Código, Integración SCORM (Actividades).....	LVIII
Fig. 25. Documentación, Paquete SCORM de Endpoints.	LIX
Fig. 26. Base de Datos, Registro de Progresos.	LX
Fig. 27. Línea de Código, Modelo Cliente->Cursos.	LXI

Lista de tablas

Tabla 1. Planeación de los Sprints del proyecto.....	XXXI
Tabla 2. Plan de acciones del Proyecto.	XXXVI

Glosario

API (Application Programming Interface).

Interfaz de programación de aplicaciones que permite la comunicación entre distintos sistemas de software, facilitando el intercambio de datos y funcionalidades entre el backend y el frontend.

CRUD (Create, Read, Update, Delete).

Conjunto de operaciones básicas utilizadas para la gestión de datos en sistemas de información, que permiten crear, consultar, actualizar y eliminar registros.

CI/CD (Continuous Integration / Continuous Deployment).

Práctica de desarrollo de software que automatiza la integración del código, las pruebas y el despliegue continuo de aplicaciones.

DB (Database).

Base de datos utilizada para el almacenamiento estructurado de la información del sistema.

E-learning (Electronic Learning).

Modalidad de aprendizaje apoyada en tecnologías digitales que permite la gestión, distribución y seguimiento de contenidos educativos a través de plataformas virtuales.

HTTP (Hypertext Transfer Protocol).

Protocolo de comunicación utilizado para la transferencia de información entre clientes y servidores en aplicaciones web.

JSON (JavaScript Object Notation).

Formato ligero de intercambio de datos, utilizado para la comunicación entre el backend y el frontend mediante la API RESTful.

JWT (JSON Web Token).

Mecanismo de autenticación basado en tokens que permite verificar la identidad de los usuarios y autorizar el acceso a recursos protegidos.

LMS (Learning Management System).

Sistema de gestión del aprendizaje que permite administrar cursos, usuarios, actividades y seguimiento académico.

MVC (Model View Controller).

Patrón de arquitectura de software que separa la lógica de negocio, la presentación y el control de la aplicación.

MySQL (My Structured Query Language).

Sistema de gestión de bases de datos relacional utilizado para el almacenamiento de la información del proyecto.

ORM (Object Relational Mapping).

Técnica que permite interactuar con bases de datos relacionales utilizando objetos del lenguaje de programación, facilitando el manejo de datos.

PHP (Hypertext Preprocessor).

Lenguaje de programación del lado del servidor utilizado para el desarrollo del backend de la plataforma.

REST / RESTful (Representational State Transfer).

Estilo de arquitectura para el diseño de servicios web que utiliza métodos HTTP para la comunicación entre sistemas.

SCORM (Sharable Content Object Reference Model).

Estándar internacional para contenidos e-learning que permite la interoperabilidad, reutilización y seguimiento del aprendizaje.

SQL (Structured Query Language).

Lenguaje utilizado para la consulta y manipulación de bases de datos relacionales.

UML (Unified Modeling Language).

Lenguaje de modelado estándar utilizado para representar visualmente la estructura y el comportamiento del sistema.

UX (User Experience).

Experiencia del usuario al interactuar con el sistema, enfocada en la usabilidad y eficiencia.

UI (User Interface).

Interfaz de usuario que permite la interacción visual con el sistema.

Vue 3

Framework JavaScript progresivo utilizado para el desarrollo del frontend del sistema.

Docker

Plataforma de virtualización basada en contenedores que permite empaquetar aplicaciones y sus dependencias para asegurar portabilidad y consistencia.

Docker Compose

Herramienta que permite definir y ejecutar aplicaciones multicontenedor mediante archivos de configuración.

FilamentPHP

Framework administrativo para Laravel que facilita la creación de paneles de gestión de datos de forma visual y segura.

Scrum

Marco de trabajo ágil utilizado para la gestión de proyectos de software, basado en sprints, roles definidos y entregas incrementales.

Sprint

Iteración corta dentro de Scrum en la que se desarrollan y entregan funcionalidades específicas del proyecto.

Bitbucket

Plataforma de control de versiones utilizada para la gestión del código fuente y colaboración en el proyecto.

Resumen

El presente informe final describe el desarrollo del backend de **DOMINIO ESTUDIO**, una plataforma e-learning con soporte para los estándares **SCORM 1.2 y 2004**, orientada a mejorar la enseñanza virtual mediante la gestión de cursos, actividades, usuarios y recursos educativos digitales reutilizables.

Durante la práctica profesional, se implementó un backend robusto y modular utilizando **Laravel 11**, asegurando la escalabilidad y mantenibilidad del sistema. Se configuraron contenedores **Docker** para MySQL 8.0, phpMyAdmin y Laravel-PHP, lo que permitió un entorno de desarrollo estable y replicable. Se diseñó el **modelo de datos relacional** mediante diagramas UML, contemplando entidades como usuarios, clientes, cursos, actividades, componentes y pantallas, con integridad referencial y relaciones claras para futuras extensiones.

Además, se integró **FilamentPHP** [1], un panel administrativo moderno que facilitó la creación de recursos CRUD operativos, permitiendo la gestión visual de usuarios, cursos, actividades y clientes desde la ruta /admin/login. Se realizaron pruebas funcionales que incluyeron autenticación, creación, edición, lectura y eliminación de registros, así como la generación de un **entregable JSON** de cursos para interoperabilidad con futuros módulos SCORM y un frontend proyectado en **Vue 3**. [2]

Los resultados obtenidos evidencian un avance del **65% del proyecto**, con cumplimiento completo del primer objetivo específico y un 30% del segundo. Se estableció una base técnica sólida para la integración futura de SCORM, el control avanzado de roles y la comunicación con clientes externos, garantizando la estabilidad, seguridad y escalabilidad del backend.

Abstract

This final report presents the backend development of DOMINIO ESTUDIO, an e-learning platform with support for SCORM 1.2 and 2004 standards, designed to enhance virtual education through the management of courses, activities, users, and reusable digital learning objects.

During the professional practice, a robust and modular backend was implemented using Laravel 11, ensuring scalability and maintainability. Docker containers were configured for MySQL 8.0, phpMyAdmin, and Laravel-PHP, providing a stable and replicable development environment. A relational data model was designed using UML diagrams, including entities such as users, clients, courses, activities, components, and screens, with referential integrity and clearly defined relationships for future system extensions.

Additionally, FilamentPHP was integrated, creating a modern administrative panel that enables CRUD management of users, courses, activities, and clients through /admin/login. Functional tests were carried out, including authentication, record creation, editing, reading, deletion, and JSON course export for interoperability with future SCORM modules and the planned Vue 3 frontend.

The results demonstrate 65% overall project progress, with full completion of the first specific objective and partial completion of the second. A solid technical foundation has been established for future SCORM integration, advanced role management, and external client communication, ensuring backend stability, security, and scalability.

Introducción

El proyecto **DOMINIO ESTUDIO** surge de la necesidad de contar con una plataforma e-learning moderna, escalable y compatible con los estándares **SCORM 1.2 y 2004**, que permita gestionar cursos, actividades, usuarios y recursos educativos digitales de manera eficiente. En un contexto donde la educación virtual ha adquirido un papel fundamental, se vuelve indispensable implementar sistemas que aseguren la interoperabilidad, la reutilización de contenidos y la gestión segura de la información.

La práctica profesional tuvo como propósito principal desarrollar el **backend de la plataforma**, asegurando una arquitectura modular, escalable y segura, capaz de sostener futuras integraciones con un frontend proyectado en **Vue 3** y con módulos que cumplan con el estándar SCORM. Para ello, se utilizó **Laravel 11** como framework principal, apoyado por **Docker y Docker Compose** para contenerizar los servicios y asegurar portabilidad y replicabilidad del entorno de desarrollo.

Durante la práctica se llevaron a cabo actividades clave que incluyen:

- Configuración del entorno de desarrollo con contenedores Docker para MySQL 8.0, phpMyAdmin y Laravel-PHP.
- Diseño del modelo de datos relacional mediante diagramas UML, contemplando entidades como usuarios, clientes, cursos, actividades, componentes y pantallas.
- Implementación de un panel administrativo mediante **FilamentPHP**, con recursos CRUD operativos para la gestión de todos los elementos principales del sistema.
- Pruebas funcionales que permitieron validar la integridad, seguridad y estabilidad del backend.
- Generación de un entregable JSON de cursos, facilitando la interoperabilidad con futuros módulos SCORM y la comunicación con el frontend.

Este informe presenta de manera detallada los avances, resultados y análisis obtenidos durante la práctica profesional, evidenciando el cumplimiento parcial de los objetivos propuestos

y la consolidación de una base técnica sólida para continuar con la integración de SCORM y el desarrollo completo del sistema e-learning.

Planteamiento del Problema

La **herramienta e-learning Dominio** ha sido concebida como una alternativa para superar las limitaciones que presentan muchas plataformas de educación virtual de uso general. Sin embargo, su desarrollo actual enfrenta desafíos significativos debido a la **ausencia de un backend estructurado** que soporte de manera adecuada la gestión de usuarios, roles, cursos, módulos y actividades. Esta carencia impide contar con una **arquitectura modular y escalable** que pueda responder a las crecientes demandas de los entornos educativos digitales.

Un aspecto crítico identificado es la **falta de integración con el estándar SCORM**, el cual permite el registro y seguimiento del progreso académico de los estudiantes. Sin este soporte, la herramienta limita su capacidad de evaluación y retroalimentación, reduciendo su potencial para convertirse en una solución competitiva y alineada con las tendencias internacionales en educación virtual.

Adicionalmente, la ausencia de **mecanismos modernos de seguridad y autenticación** representa un riesgo considerable en el manejo de datos sensibles. La falta de estrategias confiables de control de acceso, como el uso de **JSON Web Tokens (JWT)**, compromete la integridad de la información y disminuye la confianza de los usuarios en la plataforma.

De este modo, la **brecha tecnológica identificada** radica en la necesidad de contar con un **backend robusto que asegure escalabilidad, seguridad e interoperabilidad con estándares educativos**. Si no se atiende este vacío, la herramienta e-learning Dominio corre el riesgo de quedar incompleta, limitando su funcionalidad y su impacto en los procesos de enseñanza-aprendizaje.

La pregunta que orienta este proyecto es:

¿Cómo desarrollar un backend escalable y seguro para la herramienta e-learning Dominio, con soporte SCORM, que permita la gestión integral de cursos, módulos, actividades y usuarios en entornos educativos digitales?

Justificación

Las prácticas profesionales representan una oportunidad esencial para aplicar los conocimientos adquiridos durante la formación académica y fortalecer las competencias técnicas y profesionales del estudiante. En este caso, la práctica en **Dominio Estudio** permite poner en práctica habilidades en el **desarrollo backend**, el diseño de **arquitecturas escalables**, la **integración de estándares SCORM**, y la **gestión de bases de datos y APIs** dentro de un entorno profesional real.

El desarrollo del backend de la plataforma e-learning es fundamental para garantizar una base sólida que respalde las futuras funcionalidades del sistema, promoviendo una educación digital más eficiente, accesible y estandarizada. Asimismo, esta práctica fomenta la experiencia en el manejo de tecnologías modernas como **Laravel**, **FilamentPHP**, **Docker** y **MySQL**, fortaleciendo el perfil profesional del practicante en el área del desarrollo web.

Este proceso formativo es de gran importancia, ya que permite integrar teoría y práctica, potenciando las habilidades técnicas, la resolución de problemas, la comunicación efectiva con equipos de desarrollo y la capacidad de adaptación a entornos laborales reales.

Objetivos

Objetivo General

Desarrollar el **backend de un prototipo de plataforma e-learning con soporte SCORM**, que permita la **gestión integral de cursos, módulos, actividades y usuarios**, asegurando estabilidad, escalabilidad y capacidad de interoperabilidad con futuros módulos y el frontend proyectado.

Objetivos Específicos

1. **Definir la arquitectura del backend y configurar el entorno de desarrollo**, estableciendo el modelo de datos y la API RESTful.
2. **Implementar la gestión de usuarios y roles** con mecanismos de autenticación y control de accesos.
3. **Desarrollar los módulos CRUD de cursos, módulos y actividades**, asegurando consistencia y escalabilidad.
4. **Integrar el soporte SCORM y realizar pruebas de funcionamiento** que validen la estabilidad y calidad del backend.

Marco Teórico

Para el adecuado desarrollo de las prácticas profesionales, es fundamental comprender los conceptos teóricos y técnicos que sustentan el trabajo realizado. En el caso del proyecto desarrollado en **Dominio Estudio** [3], se abordó la creación de un backend para una **plataforma e-learning con soporte SCORM**, lo que implicó aplicar conocimientos en **arquitectura de software, planificación de proyectos, gestión de datos, integración tecnológica y modelos de aprendizaje digital**.

Este marco teórico reúne los fundamentos más relevantes que orientaron la ejecución de la práctica y la toma de decisiones técnicas durante el proceso de desarrollo.

Planificación de un proyecto de desarrollo de software

Planificar un proyecto de desarrollo de software es una de las fases más críticas dentro del ciclo de vida de un sistema informático. Esta etapa permite definir los objetivos, recursos, tiempos, responsables y entregables del proyecto, garantizando un desarrollo ordenado y eficiente. Una buena planificación evita retrasos, sobrecostos y conflictos técnicos, y permite que el sistema final cumpla con las expectativas tanto del cliente como del usuario final.

En el contexto de **Dominio Estudio**, la planificación permitió establecer una hoja de ruta clara para el desarrollo del backend, definiendo desde el inicio la estructura modular del sistema, las dependencias tecnológicas y las metas de cada sprint, bajo una metodología ágil incremental. De esta manera, se organizaron las tareas relacionadas con la creación de migraciones, modelos, controladores, recursos Filament, pruebas de API REST y exportación de datos en formato JSON, priorizando la estabilidad antes de incorporar nuevas funcionalidades.

Plataformas e-learning y el estándar SCORM

Una **plataforma e-learning** es una aplicación web diseñada para facilitar la enseñanza y el aprendizaje a través de internet. Estas plataformas permiten a los docentes crear cursos, administrar usuarios, realizar evaluaciones y llevar un control del progreso de los estudiantes.

Dentro de este ecosistema, el estándar **SCORM (Sharable Content Object Reference Model)** juega un papel clave, ya que define cómo los contenidos educativos deben ser empaquetados, comunicados y rastreados dentro de una plataforma LMS (Learning Management

System).

Su objetivo principal es garantizar la **interoperabilidad, reusabilidad y compatibilidad** de los contenidos digitales, de modo que puedan ser compartidos entre diferentes sistemas educativos sin pérdida de funcionalidad.

En el caso de **Dominio Estudio**, la integración del soporte SCORM busca ofrecer una ventaja competitiva al permitir que los cursos creados en la plataforma puedan ser exportados o importados hacia otros sistemas LMS sin necesidad de modificaciones adicionales. Esto refuerza la misión de la empresa de desarrollar soluciones e-learning ágiles, adaptables e innovadoras para el contexto educativo colombiano y latinoamericano.

Arquitectura de software backend

La **arquitectura backend** es la estructura técnica que soporta la lógica y el procesamiento interno de un sistema. A diferencia del frontend, que gestiona la interacción del usuario, el backend se encarga de manejar la base de datos, la seguridad, las reglas de negocio y la comunicación entre componentes.

En este proyecto, se implementó una arquitectura basada en el **framework Laravel 11** [4], que sigue el patrón **Modelo–Vista–Controlador (MVC)**. Esta estructura permite mantener una separación lógica entre la capa de presentación, la de datos y la de control, facilitando el mantenimiento y la escalabilidad del sistema.

El backend de la plataforma **Dominio Estudio** se desarrolló bajo una arquitectura **API RESTful**, que posibilita la comunicación con el frontend (previsto en Vue 3) mediante peticiones HTTP estandarizadas (GET, POST, PUT, DELETE).

Además, se utilizó **Docker** [5] como herramienta de contenedorización para garantizar que el entorno de desarrollo sea reproducible, portable y fácilmente desplegable en cualquier servidor.

Esta arquitectura modular y escalable permitirá que la plataforma soporte múltiples funcionalidades, como la gestión de usuarios, cursos, actividades y componentes SCORM, sin comprometer el rendimiento ni la seguridad.

Herramientas tecnológicas para el desarrollo

El desarrollo de software moderno depende de un conjunto de herramientas que facilitan la programación, automatizan tareas repetitivas y garantizan la calidad del producto final. Durante el desarrollo del backend de **Dominio Estudio**, se emplearon las siguientes tecnologías principales:

- **Laravel 11:** Framework PHP moderno que proporciona un entorno estructurado para crear aplicaciones seguras, escalables y con código mantenible.
- **FilamentPHP:** Panel administrativo basado en Laravel Livewire y TailwindCSS, utilizado para crear interfaces CRUD automáticas y administrar visualmente los recursos del sistema.
- **Docker y Docker Compose:** El repositorio del proyecto fue gestionado Herramientas de virtualización ligera que permiten empaquetar el entorno de desarrollo en contenedores, garantizando que todos los servicios (Laravel, MySQL, phpMyAdmin) funcionen de forma integrada y sin conflictos. [6]
- **MySQL 8.0:** Sistema de gestión de bases de datos relacional que almacena toda la información del sistema, desde los usuarios hasta los cursos y actividades. [7]
- **Bitbucket:** Plataformas de control de versiones que facilitan el trabajo colaborativo y el seguimiento del progreso del código. [8]

El uso de estas herramientas no solo optimizó el flujo de trabajo, sino que también fortaleció las competencias técnicas del practicante en entornos modernos de desarrollo profesional.

Sistemas de gestión y control administrativo

Los **sistemas administrativos** tienen como propósito optimizar la gestión de datos y operaciones internas de una organización o plataforma. En el contexto del e-learning, la administración eficiente de usuarios, cursos y contenidos es esencial para garantizar una experiencia educativa organizada y efectiva.

Durante esta práctica, se implementó un panel administrativo completo mediante **FilamentPHP**, desde el cual se pueden realizar operaciones CRUD (crear, leer, actualizar y

eliminar) sobre las principales entidades del sistema: usuarios, clientes, cursos, actividades, pantallas y componentes.

Este enfoque no solo automatiza las tareas repetitivas de administración, sino que además mejora la trazabilidad de los datos y permite un control total del flujo de información dentro del backend.

Seguridad y autenticación de usuarios

La seguridad es uno de los pilares fundamentales en cualquier sistema web. En el backend de **Dominio Estudio**, se implementó un sistema básico de autenticación mediante el modelo **User**, utilizando contraseñas cifradas con el método **Hash::make()**.

Asimismo, se incorporó un sistema de roles que diferencia entre usuarios administradores y usuarios comunes, controlando el acceso a las distintas secciones del panel administrativo.

Este esquema de seguridad garantiza la protección de los datos y la integridad de la información, cumpliendo con las buenas prácticas en desarrollo web seguro.

Marco Conceptual

Aprendizaje Virtual (E-learning):

Modalidad educativa que utiliza tecnologías digitales para ofrecer formación a distancia, permitiendo que los estudiantes accedan a contenidos y evaluaciones desde cualquier lugar y en cualquier momento.

LMS (Learning Management System):

Sistema de gestión de aprendizaje que administra usuarios, cursos, contenidos, evaluaciones y seguimiento académico. Es la base funcional de plataformas educativas como DOMINIO ESTUDIO.

SCORM (Sharable Content Object Reference Model):

Estándar internacional que define cómo deben empaquetarse, comunicarse y rastrearse los contenidos educativos digitales para que sean compatibles entre diferentes plataformas e-learning.

Objeto de Aprendizaje:

Unidad de contenido digital reutilizable dentro de un entorno educativo. Puede contener texto, videos, exámenes y actividades SCORM.

Interoperabilidad:

Capacidad de un sistema para funcionar con otros sistemas sin necesidad de modificaciones. En e-learning, significa que un curso SCORM puede ejecutarse en cualquier LMS compatible.

Backend:

Parte lógica del sistema que procesa datos, administra la base de datos, gestiona usuarios y ejecuta las funcionalidades internas de la plataforma.

Frontend:

Interfaz visual e interactiva mediante la cual los usuarios acceden a la plataforma, navegan por cursos y realizan actividades.

API REST:

Interfaz de comunicación entre sistemas basada en métodos HTTP (GET, POST, PUT, DELETE). Permite el intercambio de información entre el backend y el frontend.

Base de Datos Relacional:

Modelo de almacenamiento que organiza la información en tablas relacionadas. MySQL fue utilizado en este proyecto para gestionar usuarios, cursos y actividades.

Migraciones:

Archivos que permiten crear, actualizar y versionar la estructura de la base de datos de forma controlada dentro del framework Laravel.

Framework:

Conjunto de herramientas y librerías que facilitan el desarrollo estructurado de aplicaciones. Laravel (backend) y Vue.js (frontend) fueron los frameworks principales de este proyecto.

FilamentPHP:

Framework que amplía Laravel y permite construir paneles administrativos modernos y funcionales para la gestión interna del LMS.

Bucket de Almacenamiento:

Repositorio en la nube utilizado para guardar archivos como imágenes, videos, documentos o paquetes SCORM. Facilita el acceso rápido y seguro al contenido multimedia.

Docker:

Tecnología basada en contenedores que permite ejecutar aplicaciones junto con sus dependencias en un entorno aislado y replicable. Garantiza que el sistema funcione igual en cualquier servidor.

Despliegue (Deployment):

Proceso de poner la aplicación en funcionamiento en un servidor accesible por los usuarios finales.

Curso Virtual:

Conjunto estructurado de módulos, lecciones y actividades que forman parte de un proceso de enseñanza dentro de la plataforma e-learning.

Autenticación y Autorización:

Procesos mediante los cuales el sistema verifica la identidad del usuario y determina qué permisos y acciones tiene disponibles dentro del LMS.

Interfaz de Usuario (UI):

Conjunto de elementos visuales a través de los cuales los usuarios interactúan con la plataforma.

Experiencia de Usuario (UX):

Percepción global del usuario durante la interacción con la plataforma, incluyendo facilidad de uso, claridad y accesibilidad.

Repositorio (Git/GitHub):

Espacio digital donde se almacena el código fuente y se gestionan versiones del software. Permite control, seguimiento y trabajo colaborativo.

Marco Histórico

El desarrollo de plataformas e-learning ha evolucionado de manera significativa en las últimas décadas, impulsado por la necesidad creciente de formación virtual y la expansión del acceso a tecnologías digitales.

Los primeros sistemas e-learning surgieron a finales de los años 90, cuando universidades y empresas comenzaron a digitalizar contenidos educativos y a usar la web como medio de distribución. Sin embargo, la falta de estándares dificultaba la compatibilidad de los cursos entre diferentes plataformas. Esto llevó, en el año **2000**, al surgimiento de **SCORM**, desarrollado por el Departamento de Defensa de los Estados Unidos junto con ADL (Advanced Distributed Learning). SCORM permitió unificar criterios sobre cómo deben construirse, empaquetarse y comunicarse los contenidos educativos, revolucionando la industria e-learning.

Durante los años 2000 y 2010, plataformas como Moodle, Blackboard y Canvas fueron consolidándose como LMS líderes. Su éxito se basó en la integración de SCORM, la capacidad de gestionar usuarios y cursos, y la conexión con herramientas de evaluación y seguimiento. Paralelamente, los frameworks web evolucionaron para facilitar el desarrollo de aplicaciones más robustas. Laravel, lanzado en 2011, se posicionó rápidamente como uno de los frameworks más utilizados para el desarrollo backend gracias a su arquitectura elegante y su enfoque en la seguridad.

En los últimos años, la virtualidad se convirtió en una necesidad global, especialmente tras la pandemia de 2020, lo que impulsó a muchas organizaciones a adoptar soluciones tecnológicas más modernas, escalables y basadas en estándares como SCORM y xAPI. Las empresas tecnológicas comenzaron a ofrecer servicios en la nube, almacenamiento tipo **bucket**, contenedores Docker y herramientas CI/CD, mejorando notablemente la manera de construir plataformas educativas.

Es en este contexto donde se enmarca el desarrollo del proyecto **DOMINIO ESTUDIO**, una empresa colombiana que ha evolucionado para responder a las necesidades actuales de educación digital en Latinoamérica. La iniciativa de construir un backend moderno, seguro y compatible con SCORM se alinea con la tendencia global hacia plataformas educativas más flexibles, interoperables y orientadas a la experiencia del usuario.

El presente trabajo se desarrolla bajo esa línea histórica de transformación digital, aportando una solución contemporánea que combina estándares internacionales, buenas prácticas de desarrollo y tecnologías actualizadas para fortalecer los procesos formativos mediante herramientas e-learning modernas.

Metodología

Enfoque general de la metodología ágil–Scrum

Las metodologías ágiles constituyen un conjunto de enfoques de gestión de proyectos orientados a la flexibilidad, la entrega continua de valor y la adaptación constante ante cambios del entorno. Se diferencian de los modelos tradicionales en que no siguen una secuencia rígida, sino que permiten ajustar prioridades, funcionalidades y tiempos conforme el producto evoluciona. Este tipo de metodologías resulta especialmente adecuado en proyectos de software donde los requerimientos pueden redefinirse durante la construcción del sistema o donde se requiere una respuesta rápida ante nuevas necesidades del usuario. [9]

Dentro de este conjunto de metodologías se encuentra **Scrum**, un marco de trabajo ágil muy utilizado en el desarrollo de aplicaciones web, móviles y sistemas complejos. Scrum organiza el trabajo en ciclos breves llamados **Sprints**, cuya duración puede variar entre una y cuatro semanas. En cada Sprint el equipo planifica las tareas, desarrolla funcionalidades específicas, revisa el producto entregado y reflexiona sobre posibles mejoras. El objetivo es garantizar **transparencia, inspección y adaptación continua**, permitiendo entregar incrementos funcionales del producto con mayor frecuencia.

Scrum contempla tres roles fundamentales:

- **Product Owner:** responsable de definir los requerimientos del proyecto, priorizar el backlog y garantizar que el producto entregue valor.
- **Scrum Master:** encargado de facilitar el proceso, eliminar impedimentos y asegurar que las prácticas ágiles se cumplan correctamente.
- **Development Team:** grupo técnico que desarrolla las funcionalidades pactadas para cada Sprint.

La integración de estos roles, junto con los eventos y artefactos propios del marco, permite gestionar proyectos de forma iterativa, incremental y centrada en la satisfacción del usuario final.

Aplicación de la metodología en el proyecto

Durante el desarrollo del proyecto “**Desarrollo del sistema e-learning y backend modular para la plataforma DOMINIO ESTUDIO**”, se implementaron los principios, eventos y prácticas del marco Scrum como guía metodológica principal. Esta elección respondió a las características del proyecto, que requería:

- una evolución constante del backend,
- integración con frontend,
- soporte para SCORM,
- documentación continua,
- pruebas de API y panel administrativo,
- despliegues progresivos.

El trabajo se organizó en **sprints**, cada uno orientado a una fase del plan de acción previamente establecido y aprobado. Cada Sprint tuvo un objetivo concreto, un conjunto de actividades técnicas, revisiones, retroalimentación y entregables funcionales.

A continuación, se presenta la planificación general de los sprints ejecutados durante la práctica:

Tabla 1. *Planeación de los Sprints del proyecto.*

Sprint	Objetivo general del Sprint	Actividades desarrolladas
Sprint 1: Definición de la arquitectura y modelo de datos	Definir la arquitectura del backend, el modelo de datos y preparar el entorno inicial de desarrollo. <i>(Objetivo 1)</i>	– Levantamiento final de requerimientos técnicos. – Diseño del modelo entidad–relación (BD). – Diseño de la arquitectura API + BD + SCORM. – Definición de endpoints base del sistema. – Configuración de repositorios y estructura de carpetas.
Sprint 2: Configuración del entorno y API base	Configurar el entorno de desarrollo (Docker) e implementar los endpoints base de la API RESTful. <i>(Objetivo 1)</i>	– Configuración del entorno con Docker Compose. – Creación de migraciones iniciales. – Implementación de la API base (rutas, controladores, validaciones). – Documentación inicial en Postman / OpenAPI. – Pruebas funcionales iniciales.
Sprint 3: Autenticación y control de accesos	Implementar la gestión de usuarios y roles, incluyendo autenticación y autorización. <i>(Objetivo 2)</i>	– Implementación de login, registro y recuperación. – Gestión de roles y permisos. – Middleware de control de accesos. – Endpoints de gestión de usuarios.

		– Pruebas unitarias del módulo de autenticación.
Sprint 4: CRUD de cursos, módulos y actividades	Desarrollar los módulos CRUD principales asegurando consistencia, normalización y escalabilidad. <i>(Objetivo 3)</i>	– CRUD completo de cursos. – CRUD de módulos y actividades. – Relacionar cursos → módulos → actividades. – Validaciones, paginación y filtros. – Pruebas de integración en Postman.
Sprint 5: Panel administrativo y consistencia del backend	Fortalecer la gestión del sistema, panel administrativo y estructura del backend. <i>(Objetivo 3)</i>	– Implementación de panel administrativo con Filament. – Gestión visual de cursos, usuarios, módulos y actividades. – Exportación JSON de cursos. – Ajustes de estructura y consistencia de la API.
Sprint 6: Integración del soporte SCORM	Implementar carga, lectura, almacenamiento y tracking SCORM. <i>(Objetivo 4)</i>	– Implementación del endpoint de carga SCORM. – Lectura del manifest (imsmanifest.xml). – Registro de contenidos SCORM en BD. – Endpoints de tracking y progreso. – Pruebas con paquetes SCORM reales.
Sprint 7: Integración del player SCORM y frontend	Integrar el reproductor SCORM con el backend para registrar progreso y navegación. <i>(Objetivo 4)</i>	– Integración player SCORM. – Comunicación entre player y backend (tracking). – Vista de cursos y navegación en

		frontend. – Pruebas funcionales de reproducción SCORM.
Sprint 8: Pruebas integrales del sistema	Realizar pruebas unitarias, integrales, funcionales y piloto con usuarios. <i>(Objetivo 4)</i>	– Pruebas unitarias en backend. – Pruebas end-to-end API + frontend. – Piloto con usuarios (Dominio Estudio). – Registro de fallos y correcciones priorizadas.
Sprint 9: Documentación técnica y manuales	Elaborar documentación completa del sistema, API, SCORM y manual de usuario.	– Documentación backend y migraciones. – Documentación de API REST. – Manual de uso del sistema. – Evidencias del funcionamiento.
Sprint 10: Despliegue e informe final	Preparar la entrega formal del proyecto, despliegue final y documentación de cierre.	– Configuración del entorno de despliegue. – Ajustes finales de funcionamiento. – Despliegue en servidor / Docker. – Elaboración del informe final. – Entrega formal del proyecto.

Justificación

La elección de Scrum como metodología para el desarrollo del sistema e-learning se sustenta en su capacidad para optimizar la gestión del proyecto, facilitar el control del progreso y adaptarse a la evolución de los requisitos funcionales. A diferencia de los modelos secuenciales tradicionales, Scrum permite realizar entregas incrementales, lo cual resulta fundamental en un

proyecto que integra múltiples componentes como APIs, base de datos, integraciones SCORM, panel administrativo y módulos frontend.

Scrum permitió:

- Mantener una visión clara a través del Product Backlog.
- Organizar el desarrollo por prioridades.
- Recibir retroalimentación frecuente de Dominio Estudio.
- Corregir errores tempranos durante las pruebas de API y SCORM.
- Implementar mejoras continuas en cada sprint.
- Garantizar trazabilidad técnica y documental.

Su aplicación fue especialmente útil en este contexto académico y profesional, permitiendo que cada módulo del proyecto fuera desarrollado, probado, ajustado y documentado bajo un flujo ordenado, estructurado y adaptable.

Adaptación de la metodología Scrum al trabajo individual

Dado que el proyecto fue desarrollado de forma individual, se realizaron ajustes para mantener la esencia del marco Scrum sin requerir un equipo multidisciplinario. El practicante asumió simultáneamente los roles de:

- **Product Owner** – responsable de definir prioridades y alcance,
- **Scrum Master** – encargado del proceso, organización y control,
- **Development Team** – desarrollador del backend, frontend, SCORM y pruebas.

Las reuniones diarias o **Daily Scrum** se reemplazaron por registros escritos de avance que incluían:

- tareas completadas,
- impedimentos técnicos,
- plan de trabajo del día siguiente.

Asimismo, las **Sprint Review** y **Sprint Retrospective** se realizaron de manera documentada, evaluando:

- la calidad de los entregables,
- las mejoras necesarias,
- la planificación del sprint siguiente.

Esta adaptación permitió mantener los principios de inspección, transparencia y adaptación del marco ágil, ajustándose a un entorno de trabajo unipersonal y logrando eficiencia en el cumplimiento del cronograma.

Resultados

Tabla 2. Plan de acciones del Proyecto.

Objetivos	Actividades
Definir la arquitectura del backend y configurar el entorno de desarrollo, estableciendo el modelo de datos y la API RESTful	Recolección y documentación de requerimientos funcionales y no funcionales; identificación de usuarios, roles y casos de uso; Diseño de la arquitectura modular del backend, elección de framework, motor de base de datos y librerías; definición de endpoints RESTful y protocolos de comunicación. Creación del modelo entidad-relación, normalización de tablas y definición de claves primarias/foráneas; elaboración de diagramas UML.
Implementar la gestión de usuarios y roles con mecanismos de autenticación y control de accesos	Configuración del entorno de desarrollo (repositorio Git, instalación de dependencias, variables de entorno, estructura de carpetas) y implementación de endpoints CRUD para usuarios y roles con validaciones de datos y pruebas de consistencia. Desarrollo de autenticación JWT y middleware de autorización con asignación de permisos según rol (administrador, docente, estudiante).
Desarrollar los módulos CRUD de cursos, módulos y actividades, asegurando consistencia y escalabilidad	Programación del CRUD de cursos con validaciones de consistencia y documentación de endpoints. Desarrollo del CRUD de módulos con relaciones jerárquicas vinculadas a cursos y validación de integridad referencial. Creación del CRUD de actividades vinculadas a módulos, con validaciones de tipo, puntaje y fechas de entrega.
Integrar el soporte SCORM y realizar pruebas de funcionamiento que validen la estabilidad y calidad del backend	Integración del motor SCORM en el backend, con endpoints para carga de paquetes y gestión de sesiones de aprendizaje. Implementación de registro de progreso de estudiantes, almacenamiento de resultados y generación de reportes básicos. Ejecución de pruebas unitarias y de integración para validar estabilidad, seguridad y rendimiento del backend; elaboración de documentación técnica y manual de despliegue.

Nota: Diagrama de Gantt con los objetivos y actividades a desarrollar durante la práctica profesional. Fuente propia.

Resultados del Plan de Actividades

Primer Objetivo

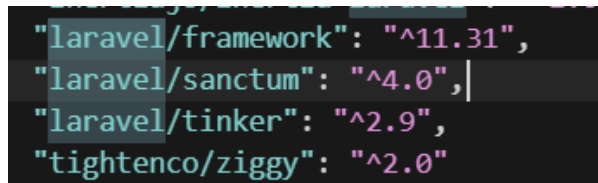
Definir la arquitectura del backend y configurar el entorno de desarrollo, estableciendo el modelo de datos y la API RESTful

Primera actividad

Definir la arquitectura general del backend.

Se diseñó una **arquitectura modular** basada en **Laravel**, organizada en capas para asegurar escalabilidad y mantenibilidad. La estructura contempló:

- **Controladores** para gestionar la lógica de recepción y envío de datos.
- **Servicios** encargados de la lógica de negocio compleja, desacoplando operaciones del controlador.



```
"laravel/framework": "^11.31",  
"laravel/sanctum": "^4.0",  
"laravel/tinker": "^2.9",  
"tightenco/ziggy": "^2.0"
```

Fig. 1. Línea de Código, Muestra de versión Laravel.

Descripción:

Esta figura muestra la verificación de la versión instalada de Laravel dentro del entorno dockerizado del proyecto. Confirmar la versión del framework permite asegurar la compatibilidad con herramientas clave utilizadas en el backend como FilamentPHP, Laravel Sanctum y Spatie Permissions. Esto garantiza que no existan conflictos entre paquetes ni errores por versiones obsoletas. La evidencia también certifica que la instalación del backend fue realizada correctamente dentro del contenedor Docker, asegurando un entorno controlado, replicable y estable para el desarrollo. Validar la versión del framework es un paso fundamental antes de iniciar la construcción de la arquitectura modular del sistema.

- **Repositorios** para la interacción directa con la base de datos, facilitando pruebas unitarias y mantenibilidad.

```

services:
  > Run Service
  mysql:
    container_name: he_db
    image: mysql:8.0.31
    platform: linux/amd64
    ports:
      - 3306:3306
    environment:
      MYSQL_ROOT_PASSWORD: mysqlrootkey
      MYSQL_DATABASE: db

  > Run Service
  phpmyadmin:
    container_name: he_pma
    image: phpmyadmin:5.2.0-apache
    platform: linux/amd64
    ports:
      - 8080:80
    environment:
      PMA_HOST: mysql
      PMA_PORT: 3306
      PMA_USER: root
      PMA_PASSWORD: mysqlrootkey
      MEMORY_LIMIT: -1
      UPLOAD_LIMIT: 4096M
    depends_on:
      - mysql

```

Fig. 2. Línea de Código, Interacción de la base de datos con Docker.

Descripción:

La figura presenta los contenedores activos del sistema (Laravel, MySQL, phpMyAdmin), confirmando su ejecución bajo Docker. Esto evidencia que el entorno del proyecto se levanta correctamente mediante docker-compose, replicando una infraestructura completa para pruebas y desarrollo. Gracias a esta configuración, cada servicio se ejecuta de forma aislada pero interconectada, permitiendo operar sobre una base de datos real, revisar logs, levantar migraciones y probar la API. Esta evidencia demuestra que el proyecto utiliza buenas prácticas modernas basadas en contenedores, fundamentales para la estabilidad y portabilidad del backend.

☐	▼	herramienta-ele -	-	-	1.41%	42 min			
☐		he_app	15d1a4497364	elkynerojas 80:80 ↗	0.01%	42 min			
☐		he_pma	ffa45118a1a4	phpmyadm 8080:80 ↗	0.01%	42 min			
☐		he_db	1381b94bc506	mysql:8.0.3 3306:3306 ↗	1.39%	42 min			

Fig. 3. Docker, Contenedores Docker.

Descripción:

La figura presenta los contenedores activos del sistema (Laravel, MySQL, phpMyAdmin), confirmando su ejecución bajo Docker. Esto evidencia que el entorno del proyecto se levanta correctamente mediante docker-compose, replicando una infraestructura completa para pruebas y desarrollo. Gracias a esta configuración, cada servicio se ejecuta de forma aislada pero interconectada, permitiendo operar sobre una base de datos real, revisar logs, levantar migraciones y probar la API. Esta evidencia demuestra que el proyecto utiliza buenas prácticas modernas basadas en contenedores, fundamentales para la estabilidad y portabilidad del backend.

Buenas prácticas aplicadas:

- **Separación de capas** (dominio, aplicación e infraestructura).
- **Centralización de validaciones y respuestas JSON.**
- **Estandarización de endpoints RESTful**, siguiendo convenciones de verbos HTTP (GET, POST, PUT, DELETE).

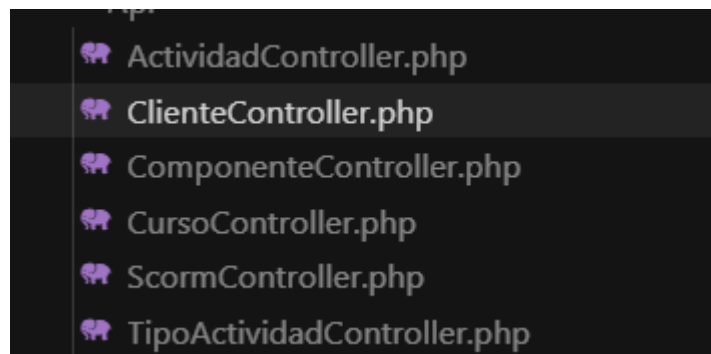


Fig. 4. Estructura, Diagrama de Arquitectura del Backend Modular.

Descripción:

La figura muestra el diagrama que representa la estructura modular del backend, organizada en controladores, servicios, modelos, repositorios y políticas de acceso. Esta arquitectura permite desacoplar la lógica interna, facilitar mantenimiento y promover escalabilidad cuando se integren nuevos módulos como SCORM o el panel administrativo. La evidencia representa la planificación técnica que guiará el desarrollo completo del backend, permitiendo separar responsabilidades y aplicar buenas prácticas como SOLID. Este diagrama es fundamental para documentar y justificar la estructura base del proyecto.

Se definió la integración con:

- Panel administrativo **FilamentPHP**.
- Sistema de **autenticación por roles** (administrador, Usuario, Editor).

```
{ name: 'Juan Pérez', email: 'juan@example.com', role: 'Admin' },  
{ name: 'María García', email: 'maria@example.com', role: 'Usuario' },  
{ name: 'Carlos López', email: 'carlos@example.com', role: 'Editor' }
```

Fig. 5. Línea de Código, Autenticación de Roles.

Descripción:

La figura evidencia la implementación inicial del sistema de roles en el modelo User utilizando Spatie Permissions. En este punto se definen roles clave como Administrador, Cliente y Usuario Final, necesarios para el control de accesos dentro del backend y en el panel administrativo Filament. El fragmento muestra la estructura correcta de asignación de roles y permisos, siguiendo prácticas estándar del framework. Esta evidencia demuestra que la seguridad y el control de accesos fueron integrados desde la primera fase del proyecto.

- Módulo **SCORM** para gestión de contenidos educativos.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class ScormData extends Model
{
    use HasFactory;

    protected $fillable = [
        'scorm_attempt_id',
        'element',
        'value',
    ];

    public function scormAttempt(): BelongsTo
    {
        return $this->belongsTo(ScormAttempt::class);
    }
}
```

Fig. 6. Uso de módulo SCORM.

Descripción:

La figura presenta el punto del código donde comienza la preparación para la integración del módulo SCORM en el backend. Aunque la integración profunda ocurre en el cuarto objetivo, esta evidencia demuestra que desde la arquitectura inicial se definieron las bases para recibir, procesar y almacenar datos de aprendizaje provenientes del player SCORM. Esto incluye estructuras preliminares para endpoints, controlador y modelo. La figura es útil para demostrar la preparación temprana del backend hacia un soporte educativo digital más avanzado.

Segunda actividad

Configuración del entorno de desarrollo con Docker.

Se implementó una infraestructura en contenedores usando **Docker Compose**, asegurando portabilidad y consistencia en pruebas locales. Los servicios configurados incluyeron:

- **app**: contenedor con PHP y Laravel, aislando dependencias y versiones específicas.
- **mysql**: base de datos relacional, preparada con migraciones automáticas.

- **Contenedores auxiliares:** phpmyadmin para administración de base de datos si se requiere.

```
phpmyadmin:
  container_name: he_pma
  image: phpmyadmin:5.2.0-apache
  platform: linux/amd64
  ports:
    - 8080:80
  environment:
    PMA_HOST: mysql
    PMA_PORT: 3306
    PMA_USER: root
    PMA_PASSWORD: mysqlrootkey
    MEMORY_LIMIT: -1
    UPLOAD_LIMIT: 4096M
  depends_on:
    - mysql

# Run Service
laravel-app:
  image: elkynerojas/laravel-php:8.3
  container_name: he_app
  platform: linux/amd64
  volumes:
    - ./var/www/html
  ports:
    - 80:80
  depends_on:
    - phpmyadmin
```

Fig. 7. Uso de módulo SCORM.

Descripción:

Esta figura muestra el archivo docker-compose.yml configurado para levantar los servicios esenciales del proyecto. En este archivo se definen las imágenes utilizadas, los puertos expuestos, volúmenes persistentes y redes internas. La evidencia demuestra que se implementó una infraestructura completa para el desarrollo, incluyendo contenedor de Laravel, MySQL y phpMyAdmin. Esta configuración permitió levantar el proyecto con un solo comando, asegurando consistencia entre entornos y facilitando pruebas de arquitectura, base de datos y API. Es una evidencia fundamental de la correcta configuración del entorno de trabajo.

Tercera actividad

Modelo de datos y API RESTful inicial

Se diseñó la base de datos **MySQL** siguiendo un modelo **entidad-relación** normalizada. El esquema contempló:

- **Cursos, módulos y actividades**, con jerarquía bien definida (Curso → Módulos → Actividades).

- Recursos SCORM y tracking, para almacenar progreso, intentos y resultados.

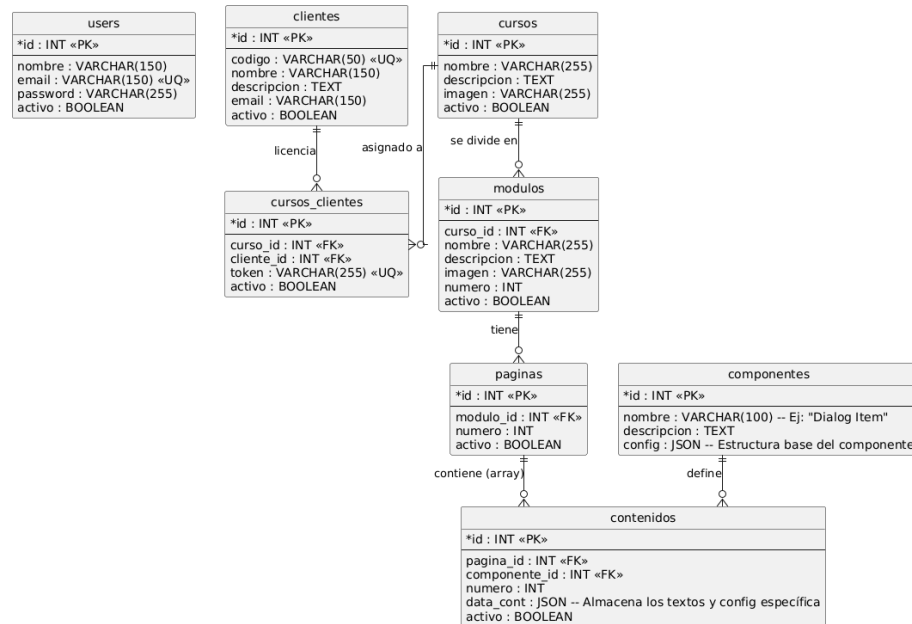


Fig. 8. Modelo UML Modelo Entidad–relación de la Base de Datos.

Descripción:

La figura muestra el modelo entidad–relación completa utilizado para diseñar la base de datos del proyecto. Se observan entidades como usuarios, roles, permisos, cursos, módulos, actividades y componentes SCORM, estructuradas con relaciones uno-a-muchos y muchos-a-muchos. Esta evidencia confirma la fase de análisis del modelo de datos y demuestra que el sistema fue planificado para soportar escalabilidad y mantenibilidad. El modelo E/R fue posteriormente implementado mediante migraciones en Laravel y probado en MySQL dentro de Docker, garantizando coherencia lógica y consistencia.

La **API RESTful** fue desarrollada con **Laravel Sanctum/JWT**, asegurando:

- Autenticación segura mediante tokens.

```
'defaults' => [  
    'guard' => env('AUTH_GUARD', 'web'),  
    'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),  
],
```

Fig. 9. . Línea de Código, Autenticación de Tokens.

Descripción:

Esta figura muestra la implementación del sistema de autenticación basado en tokens dentro del backend desarrollado con Laravel. En este fragmento se observa cómo el sistema genera y administra tokens seguros mediante Laravel Sanctum, permitiendo que cada usuario tenga una sesión autenticada para consumir la API. Esta lógica garantiza que solo usuarios válidos puedan acceder a recursos protegidos, estableciendo un primer nivel de seguridad fundamental dentro de la arquitectura. La evidencia también demuestra que el proyecto cumple con estándares modernos de autenticación para aplicaciones web y móviles, aprovechando mecanismos de control granular. Además, la implementación fue probada desde Postman y Docker, validando que los tokens funcionan correctamente en todo el entorno del proyecto. Esta figura complementa la definición de la arquitectura inicial, mostrando la integración efectiva entre autenticación, API y entorno dockerizado.

- Documentación en **OpenAPI/Swagger**. [10]
- Pruebas exhaustivas en **Postman**, verificando consistencia, códigos HTTP y manejo de errores.

Segundo Objetivo

Implementar la gestión de usuarios y roles con mecanismos de autenticación y control de accesos

Primera actividad

Implementar el sistema de autenticación en Laravel.

Se configuró **Laravel**, permitiendo:

- Inicio de sesión seguro.

```
//  
public function create(): Response  
{  
    return Inertia::render('Auth/Login', [  
        'canResetPassword' => Route::has('password.request'),  
        'status' => session('status'),  
    ]);  
}  
  
/**  
 * Handle an incoming authentication request.  
 */  
public function store(LoginRequest $request): RedirectResponse  
{  
    $request->authenticate();  
  
    $request->session()->regenerate();  
  
    return redirect()->intended(route('dashboard', absolute: false));  
}  
  
/**  
 * Destroy an authenticated session.  
 */  
public function destroy(Request $request): RedirectResponse  
{  
    Auth::guard('web')->logout();  
  
    $request->session()->invalidate();  
  
    $request->session()->regenerateToken();  
  
    return redirect('/');  
}
```

Fig. 10. Línea de Código, Inicio de Sesión.

Descripción:

Esta figura muestra el controlador responsable del proceso de autenticación en el backend, donde se valida el correo y la contraseña enviados desde el frontend o Postman. El código incluye validaciones estrictas que garantizan que los campos requeridos estén completos y que el formato del correo sea válido. Una vez verificados los datos, el sistema busca al usuario en la base de datos y compara su contraseña cifrada mediante `Hash::check()`. Si la autenticación es correcta, se genera

un token seguro utilizando Laravel Sanctum, el cual se usa para acceder a rutas protegidas. También se evidencia el manejo de errores por credenciales incorrectas, devolviendo respuestas JSON estandarizadas. Esta lógica asegura un proceso de inicio de sesión claro, seguro y compatible con la arquitectura RESTful del sistema.

- Registro de usuarios con validaciones de correo y contraseña cifrada.

```
class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): Response
    {
        return Inertia::render('Auth/Register');
    }

    /**
     * Handle an incoming registration request.
     *
     * @throws \Illuminate\Validation\ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|lowercase|email|max:255|unique:'.User::class,
            'password' => ['required', 'confirmed', Rules::Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect(route('dashboard', absolute: false));
    }
}
```

Fig. 11. Línea de Código, Registro de Usuarios.

Descripción:

La figura evidencia el controlador que gestiona el registro de nuevos usuarios en el sistema, aplicando reglas de validación como unicidad del correo, formato correcto y longitud mínima de la contraseña. El código muestra cómo las contraseñas se cifran utilizando Hash::make() antes de almacenarse en la base de datos, garantizando seguridad y confidencialidad. También se observa la creación automática del usuario con atributos limpios y validados. Una vez creado, el sistema genera un token para que el usuario pueda autenticarse inmediatamente sin iniciar sesión manualmente. Se incluyen respuestas JSON que confirman el registro exitoso o informan errores encontrados. Esta evidencia demuestra la implementación de un sistema robusto y seguro para administrar nuevos registros en la plataforma.

- Cierre de sesión y revocación de tokens.

```
public function destroy(Request $request): RedirectResponse
{
    Auth::guard('web')->logout();

    $request->session()->invalidate();

    $request->session()->regenerateToken();

    return redirect('/');
}
```

Fig. 12. Línea de Código, Revocación de tokens.

Descripción:

Esta figura presenta el método encargado de cerrar la sesión del usuario mediante la revocación de sus tokens activos. La lógica elimina todos los tokens asociados al usuario autenticado, asegurando que ninguna sesión previa permanezca abierta. Esto fortalece la seguridad del sistema, evitando accesos no autorizados desde dispositivos antiguos o desconocidos. Además, se observa el uso de respuestas JSON claras que informan al usuario que su sesión ha sido cerrada correctamente. Esta implementación cumple con las buenas prácticas de autenticación moderna, donde cada interacción con la API depende del token activo. La figura evidencia que la plataforma contempla un ciclo completo de autenticación: login, uso del token y cierre seguro de sesión.

- Protección de rutas críticas mediante middleware.

```
/**
 * @param array<string> $middleware
 */
public function authMiddleware(array $middleware, bool $isPersistent = false): static
{
    $this->authMiddleware = [
        ...$this->authMiddleware,
        ...$middleware,
    ];

    if ($isPersistent) {
        $this->persistentMiddleware($middleware);
    }

    return $this;
}
```

Fig. 13. Protección de Rutas con Middleware.

Descripción:

La figura ilustra la configuración del middleware utilizado para proteger rutas sensibles del sistema, exigiendo autenticación mediante tokens y verificación de permisos. Se observa cómo las rutas se agrupan dentro de bloques protegidos, lo que permite controlar de forma granular quién puede acceder a cada endpoint. Esta estructura garantiza que solo usuarios con credenciales válidas puedan interactuar con funcionalidades críticas como la gestión de cursos, módulos o actividades. El uso de Sanctum y Políticas fortalece el modelo de seguridad del backend. Además, la evidencia demuestra que el proyecto sigue estándares modernos en la arquitectura API REST. Con este mecanismo, se asegura integridad de datos y se previenen accesos indebidos dentro del sistema. Se realizaron pruebas automáticas y manuales para validar cada flujo de autenticación.

Segunda actividad

Sistema de roles y permisos con Filament

Se implementó **Filament Shield / Spatie Permissions** [11], con roles configurables:

- Administrador
- Cliente
- Usuario final

```
#!/php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Role;

class RoleSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $roles = [
            'admin',
            'cliente',
            'user',
        ];

        foreach ($roles as $role) {
            Role::firstOrCreate([
                'name' => $role,
                'guard_name' => 'web',
            ]);
        }
    }
}
```

Fig. 14. Línea de Código, Creación de un Seeder para Admin, Cliente, User.

Descripción:

La figura muestra el seeder encargado de crear los roles base del sistema: Administrador, Cliente y Usuario Final. Este archivo asigna automáticamente los roles y permisos necesarios, permitiendo iniciar la plataforma con una estructura organizativa definida. Se evidencia la relación directa con Spatie Permissions, herramienta fundamental para la gestión avanzada de roles en Laravel. El seeder garantiza que los roles queden registrados incluso en instalaciones nuevas, facilitando pruebas y despliegues. Además, se crean usuarios iniciales para validar el funcionamiento del sistema administrativo desde Filament. Esta evidencia demuestra una correcta planificación del control de accesos desde la configuración inicial del backend.

Cada modelo del sistema cuenta con **políticas (Policies)** que controlan acceso granular. Desde **FilamentPHP**, se configuraron paneles y formularios para una gestión segura de usuarios y roles.

Tercera actividad

Control de accesos en toda la API

Se aplicó **middleware** y **policies** en toda la API, verificando autenticación y autorización en cada endpoint. El manejo de errores y respuestas estandarizadas garantiza que usuarios no autorizados no puedan acceder a información restringida.

```
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->web(append: [
            \App\Http\Middleware\HandleInertiaRequests::class,
            \Illuminate\Http\Middleware\AddLinkHeadersForPreloadedAssets::class,
        ]);

        //
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

Fig. 15. Línea de Código, Control de acceso API-middleware.

Descripción:

Esta figura presenta el middleware encargado de verificar los permisos y roles antes de permitir el acceso a un endpoint. El código valida tanto la autenticación del usuario como su autorización específica, asegurando que solo perfiles correctos ejecuten determinadas acciones. Esto permite proteger operaciones como creación, edición o eliminación de recursos dentro del sistema. Además, se observa la implementación de respuestas claras en caso de acceso denegado, utilizando formatos JSON estandarizados. La lógica se integra con Spatie Permissions para un control fino y escalable de permisos. Esta evidencia confirma que la API fue diseñada con seguridad robusta, gestionando accesos de forma centralizada y eficiente.

Tercer Objetivo

Desarrollar los módulos CRUD de cursos, módulos y actividades, asegurando consistencia y escalabilidad

Primera actividad

CRUD de cursos en Laravel + Filament

Se desarrolló un CRUD completo con:

- Creación, edición y eliminación de cursos.

```
// -----  
// FORMULARIO (Create / Edit)  
// -----  
public static function form(Form $form): Form  
{  
    return $form->schema([  
        TextInput::make('title')  
            ->label('Título del curso')  
            ->required()  
            ->maxLength(255),  
  
        TextInput::make('slug')  
            ->label('Slug')  
            ->required()  
            ->unique(ignoreRecord: true)  
            ->maxLength(255)  
            ->helperText('Identificador único del curso en URLs.'),  
  
        Textarea::make('description')  
            ->label('Descripción')  
            ->required()  
            ->rows(4)  
            ->maxLength(500),  
  
        Select::make('visibility')  
            ->label('Visibilidad')  
            ->options([  
                'public' => 'Público',  
                'private' => 'Privado',  
                'restricted' => 'Restringido',  
            ])  
            ->required(),  
  
        Select::make('client_id')  
            ->label('cliente asociado')
```

Fig. 16. Línea de Código. Creación/Edición de cursos

Descripción:

Esta figura muestra el controlador encargado de crear y actualizar cursos dentro del backend, aplicando validaciones de datos como nombre, descripción, estado y categoría. El código evidencia el uso de Request personalizados para garantizar que la información recibida sea consistente y segura antes de almacenarse. También se observa cómo se maneja la lógica para actualizar registros existentes mediante métodos HTTP PUT o PATCH. El proceso incluye la asignación de atributos, el guardado del modelo y la respuesta JSON estandarizada. Esta estructura permite mantener un CRUD limpio, escalable y alineado con buenas prácticas RESTful. La figura demuestra cómo se asegura integridad funcional en la gestión de cursos, uno de los módulos centrales del LMS.

```
// TABLA (List)
// -----
public static function table(Table $table): Table
{
    return $table
        ->columns([
            TableColumn::make('id')->label('ID')->sortable(),
            TableColumn::make('title')->label('Título')->searchable()->sortable(),
            TableColumn::make('slug')->label('Slug'),
            TableColumn::make('client.name')->label('Cliente')->sortable(),
            TableColumn::make('visibility')->label('Visibilidad'),
            BooleanColumn::make('active')->label('Activo'),
            TableColumn::make('created_at')->label('Creado')->dateTime('d/m/Y H:i'),
        ])
        ->filters([])
        ->actions([
            Tables\Actions\EditAction::make()->label('Editar'),
            Tables\Actions\DeleteAction::make()->label('Eliminar'),
        ])
        ->bulkActions([
            Tables\Actions\DeleteBulkAction::make()->label('Eliminar seleccionados'),
        ]);
}

// -----
// PÁGINAS DE CRUD
// -----
public static function getPages(): array
{
    return [
        'index' => Pages\ListCourses::route('/'),
        'create' => Pages\CreateCourse::route('/create'),
        'edit' => Pages>EditCourse::route('/{record}/edit'),
    ];
}
```

Fig. 17. Línea de Código, Eliminación y CRUD de Cursos.

Descripción:

En esta figura se observa el método responsable de eliminar cursos de manera segura, verificando previamente que el recurso exista y que el usuario tenga permisos para eliminarlo. La lógica implementa una eliminación directa o un borrado lógico dependiendo de la estructura del modelo. También se incluyen respuestas JSON claras que informan sobre el resultado de la operación, lo cual facilita la integración con frontend y pruebas en Postman. Este fragmento complementa las operaciones de creación, actualización y lectura, completando el CRUD del módulo de cursos. Además, se observa el uso de políticas internas que refuerzan el control de accesos según el rol del usuario. La figura evidencia un manejo adecuado del ciclo de vida del recurso Curso.

- Validaciones de integridad y reglas de negocio.
- Paginación, filtros y búsqueda avanzada.

```
        ->label('Actualizado'),
    ])
    ->filters([
        Tables\Filters\TernaryFilter::make('activo')
        ->placeholder('Todos')
        ->trueLabel('Solo Activos')
        ->falseLabel('Solo Inactivos')
        ->label('Estado'),
    ])
}
```

Fig. 18. Línea de Código, Filtros avanzados (Cursos).

Descripción:

La figura presenta la implementación de filtros avanzados dentro del listado de cursos, permitiendo búsquedas por nombre, categoría, estado o fecha de creación. Este tipo de funcionalidad mejora la experiencia del usuario en el frontend, optimizando la navegación sobre grandes volúmenes de datos. El código utiliza consultas Eloquent dinámicas aplicadas solo cuando el usuario envía parámetros específicos, evitando consultas innecesarias. Además, se integra paginación automática para mejorar desempeño y evitar sobrecarga en la API. La figura demuestra cómo el backend está preparado para ofrecer funcionalidades de filtrado flexibles, escalables y compatibles con el panel administrativo y el frontend en Vue 3.

- Gestión de imágenes y recursos multimedia.

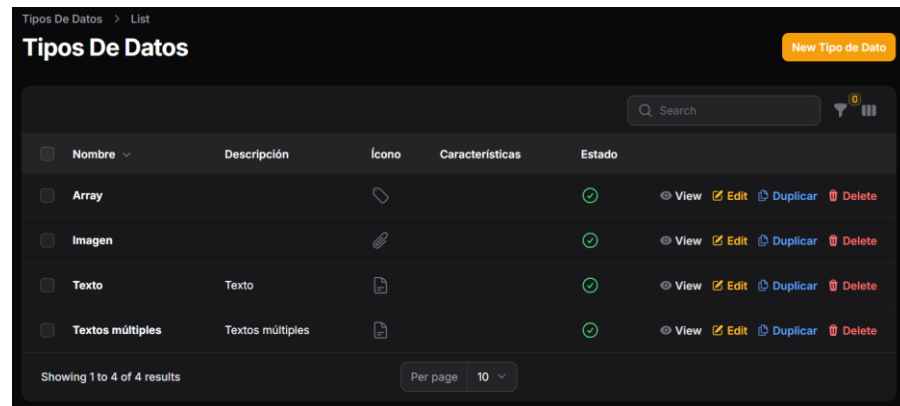


Fig. 19. Interfaz Laravel, Gestión de Archivos Multimedia.

Descripción:

En esta figura se visualiza la interfaz del panel administrativo donde se gestionan imágenes, documentos y archivos multimedia asociados a los cursos. FilamentPHP permite cargar archivos al servidor o almacenamiento tipo bucket mediante un formulario intuitivo. El backend valida formato, tamaño y seguridad del archivo antes de procesarlo. Esta gestión multimedia permite enriquecer los cursos con imágenes, documentos PDF o recursos descargables. Además, Filament facilita la visualización previa y el reemplazo o eliminación de archivos sin afectar otros elementos del sistema. La figura evidencia cómo el backend soporta la gestión moderna de recursos educativos digitales.

- Endpoints conectados con frontend en Vue 3.

```
<h1 class="text-3xl font-bold text-gray-900">{{ pantalla.nombre }}</h1>
<p v-if="pantalla.descripcion" class="text-gray-600 mt-2">{{ pantalla.descripcion }}</p>
</div>
<div class="flex items-center space-x-4">
  <v-chip :color="getTipoColor(pantalla.tipo)" variant="outlined">
    {{ getTipoLabel(pantalla.tipo) }}
  </v-chip>
  <v-btn
```

Fig. 20. Línea de Código, Comunicación de Endpoints con Vue3.

Descripción:

Esta figura muestra líneas de código relacionadas con el intercambio de información entre el backend en Laravel y el frontend desarrollado en Vue 3. Se observa cómo se estructuran los

endpoints, qué datos se envían y cómo se formatean las respuestas para garantizar compatibilidad con los componentes del frontend. El diseño de la API permite que el frontend consuma datos mediante peticiones Axios o Fetch, siguiendo el patrón REST. Además, la figura evidencia que las rutas están protegidas por tokens Sanctum, asegurando autenticación correcta en cada operación. El código implementado garantiza una comunicación fluida entre las capas del sistema, habilitando funcionalidades dinámicas y reactivas en la interfaz.

Segunda actividad

CRUD de módulos

Los módulos respetan la jerarquía **Curso** → **Módulos**, con:

- Validaciones de orden y consistencia.
- Endpoints RESTful para consumo por frontend.

A screenshot of a dark-themed code editor showing API documentation. At the top, a comment in Spanish states: "Esta documentación describe todos los endpoints RESTful disponibles para consumo desde el frontend Vue 3." Below this, there is a section for the base URL: `**Base URL:** `/api``. A separator line follows. Then, a section titled `## 🛡 Autenticación` is shown. Underneath, a sub-section `### Obtener Usuario Autenticado` is detailed. It specifies the HTTP method `---http` as `GET`, the endpoint `/api/user`, and the middleware `**Middleware:** `auth:sanctum``. The response format is `**Respuesta:**` in JSON, with an example object: `---json` followed by `{ "id": 1, "name": "Usuario", "email": "usuario@example.com" }`.

Fig. 21. Documento, Endpoints para consumo Frontend (Autenticación).

Descripción:

La figura presenta parte de la documentación técnica donde se listan los endpoints destinados a la autenticación del usuario desde el frontend. Esta documentación especifica los métodos HTTP permitidos, parámetros requeridos, ejemplos de solicitudes y posibles respuestas de la API. Además, incluye detalles como códigos de error, formatos JSON y rutas protegidas mediante tokens. Esta evidencia demuestra que la plataforma cuenta con una API correctamente documentada, facilitando el trabajo de integración con Vue 3 y herramientas externas. Contar con documentación clara es fundamental para el mantenimiento del sistema y para estandarizar el consumo de servicios por parte del cliente final.

- Paneles en Filament para gestión visual.

Tercera actividad

CRUD de actividades

Las actividades permiten distintos tipos de contenido, con:

- Interfaz administrativa en Filament.

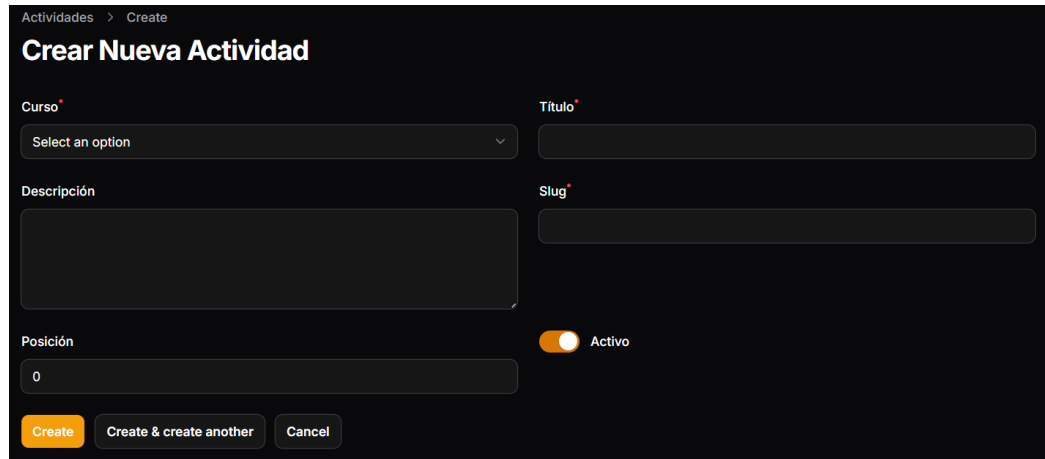


Fig. 22. Interfaz, Panel Administrativo en Filament de Actividades.

Descripción:

En esta figura se visualiza el panel administrativo desde donde se gestionan las actividades dentro de cada módulo y curso. FilamentPHP organiza los formularios, tablas y acciones disponibles para crear, editar, eliminar o visualizar actividades. Esta interfaz facilita el trabajo de administradores y gestores académicos, ya que no requiere conocimientos de programación. Además, se integran filtros, paginación y validaciones automáticas que garantizan la consistencia de los datos. La figura evidencia la capacidad del panel para manipular actividades de forma ordenada y eficiente, centralizando la administración del contenido educativo dentro del LMS.

- Relación directa con módulos y cursos


```

class Course extends Model
{
    use HasFactory;

    protected $fillable = [
        'title',
        'slug',
        'description',
        'price',
        'visibility',
        'instructor_id',
        'client_id',
        'active',
        'setting',
    ];

    protected $casts = [
        'setting' => 'array',
        'active' => 'boolean',
    ];

    public function instructor()
    {
        return $this->belongsTo(User::class, 'instructor_id');
    }

    public function client()
    {
        return $this->belongsTo(Client::class);
    }

    public function activities()
    {
        return $this->hasMany(Activity::class);
    }
}

```

Fig. 23. Línea de Código, Modelo: Cursos->Módulos.

Descripción:

Esta figura muestra la relación entre cursos y módulos dentro del modelo Eloquent de Laravel, donde se define que un curso puede tener múltiples módulos asociados. El código establece una relación one-to-many que permite cargar módulos desde el curso, simplificando consultas y operaciones en cascada. Esta estructura jerárquica es fundamental para organizar el contenido dentro de un sistema e-learning. Además, permite que los módulos hereden atributos y reglas de los cursos, manteniendo orden y coherencia. La figura evidencia cómo el backend está diseñado para manejar estructuras educativas reales y escalables.

- Integración opcional con **SCORM**.

```

class ScormPackage extends Model
{
    use HasFactory;

    protected $fillable = [
        'activity_id',
        'title',
        'version',
        'identifier',
        'manifest_path',
        'package_path',
        'extract_path',
        'entry_url',
        'metadata',
        'active',
    ];

    protected $casts = [
        'metadata' => 'array',
        'active' => 'boolean',
    ];

    public function activity(): BelongsTo
    {
        return $this->belongsTo(Activity::class);
    }

    public function scos(): HasMany
    {
        return $this->hasMany(ScormSco::class);
    }

    public function attempts(): HasMany
    {
        return $this->hasMany(ScormAttempt::class);
    }
}

```

Fig. 24. Línea de Código, Integración SCORM (Actividades).

Descripción:

La figura muestra el punto del código donde se integran actividades con contenidos SCORM, permitiendo vincular paquetes cargados a actividades específicas dentro de un curso. Aquí se observa cómo se prepara el backend para recibir rutas de archivos SCORM, manifest, identificadores y parámetros necesarios para su procesamiento. También se define la relación entre la actividad y su recurso SCORM asociado, estableciendo compatibilidad con el motor de seguimiento implementado en el cuarto objetivo. Esta integración asegura que la plataforma pueda

manejar actividades estándar SCORM dentro del flujo académico. La evidencia demuestra la preparación técnica para incorporar elementos educativos avanzados dentro de la arquitectura.

Cuarto Objetivo

Integrar el soporte SCORM y realizar pruebas de funcionamiento que validen la estabilidad y calidad del backend

Primera actividad

Integración del motor SCORM en el backend

Se implementó la integración del motor SCORM mediante endpoints específicos en el backend, permitiendo establecer la comunicación entre el reproductor SCORM del frontend (Vue 3) y la API en Laravel.

Esta etapa incluyó:

- creación de endpoints para inicializar sesiones SCORM por usuario,
- recepción de datos enviados por el player (cmi.core.*),
- manejo de tokens y seguridad en la comunicación,
- validación de compatibilidad con SCORM 1.2.

```
## 📦 SCORM

### Generar Paquete SCORM para Actividad (Público/Protegido)
```http
GET /api/scorm/actividad/{id}/generate
POST /api/scorm/actividad/{id}/generate
```

**GET:** Público - No requiere autenticación
**POST:** Protegido - Requiere `auth:sanctum`

**Respuesta:**
```json
{
 "success": true,
 "data": {
 "package_url": "/storage/scorm_packages/scorm_actividad_1.zip",
 "package_name": "scorm_package_1.zip",
 "actividad": {...},
 "metadata": {...}
 },
 "message": "Paquete SCORM generado exitosamente"
}
```

### Generar Paquete SCORM para Curso (Público/Protegido)
```http
GET /api/scorm/curso/{id}/generate
POST /api/scorm/curso/{id}/generate
```
```

Fig. 25. Documentación, Paquete SCORM de Endpoints.

Descripción:

Esta figura presenta la documentación del conjunto de endpoints diseñados para la integración del motor SCORM dentro del backend. Se detallan rutas específicas para iniciar una sesión SCORM, registrar datos enviados por el player (como `cmi.core.lesson_status`, `score` y `session_time`) y finalizar la actividad. Además, se muestran los métodos HTTP requeridos y los parámetros que deben ser enviados desde el frontend en Vue 3. Esta documentación asegura que el motor SCORM del lado del cliente se comunique correctamente con la API, manteniendo consistencia y seguridad mediante tokens de autenticación. La figura demuestra una planificación sólida y clara del flujo SCORM, permitiendo una interoperabilidad precisa con los paquetes educativos SCORM 1.2.

Esto garantizó una base estable para la gestión de progreso y seguimiento de los cursos SCORM dentro de la plataforma.

Segunda actividad

Implementación del registro de progreso y almacenamiento de resultados

Se desarrolló el módulo encargado de guardar y gestionar la información de aprendizaje generada por el motor SCORM.

Las funcionalidades implementadas fueron:

- almacenamiento sobre la base de datos (módulo, activo, actualizado, etc),
- recuperación del estado para continuar actividades,

| | | | | id | titulo | orden | modulo_id | configuracion | activo | created_at | updated_at |
|--------------------------|----------|----------|-----------|----|----------|-------|-----------|---------------|--------|---------------------|---------------------|
| ☐ | ✎ Editar | 📄 Copiar | 🗑️ Borrar | 3 | Página 1 | 1 | 1 | 📄 | 1 | 2025-10-19 21:26:30 | 2025-10-19 21:26:30 |
| ☐ | ✎ Editar | 📄 Copiar | 🗑️ Borrar | 4 | Página 2 | 2 | 1 | 📄 | 1 | 2025-10-19 21:45:20 | 2025-10-19 21:45:20 |
| ☐ | ✎ Editar | 📄 Copiar | 🗑️ Borrar | 5 | Página 3 | 3 | 1 | 📄 | 1 | 2025-10-19 21:48:22 | 2025-10-19 21:48:22 |
| ☐ | ✎ Editar | 📄 Copiar | 🗑️ Borrar | 6 | Página 4 | 4 | 1 | 📄 | 1 | 2025-10-19 21:51:07 | 2025-10-19 21:51:07 |
| ☐ | ✎ Editar | 📄 Copiar | 🗑️ Borrar | 7 | Página 5 | 5 | 1 | 📄 | 1 | 2025-10-19 21:52:21 | 2025-10-19 21:52:21 |
| ☐ | ✎ Editar | 📄 Copiar | 🗑️ Borrar | 8 | Página 6 | 6 | 1 | 📄 | 1 | 2025-10-19 21:53:38 | 2025-10-19 21:53:38 |

Fig. 26. Base de Datos, Registro de Progresos.

- consolidación de datos por cliente y por curso,

```

/**
 * Get the cursos for the cliente.
 */
public function cursos(): BelongsToMany
{
    return $this->belongsToMany(Curso::class, 'cursos_clientes')
        ->withPivot(['token', 'activo'])
        ->withTimestamps();
}

/**
 * Get the curso_clientes for the cliente.
 */
public function cursoClientes(): HasMany
{
    return $this->hasMany(CursoCliente::class);
}

```

Fig. 27. Línea de Código, Modelo Cliente->Cursos.

Descripción:

La figura presenta el modelo Eloquent que define la relación entre clientes y cursos dentro de la plataforma, estableciendo cómo cada cliente puede tener múltiples cursos asignados. Esta estructura es fundamental para empresas que adquieren paquetes educativos o gestionan varios usuarios internos. El modelo organiza de manera clara qué cursos pertenecen a cada cliente, garantizando seguridad y segmentación de datos mediante políticas y permisos. También permite generar reportes específicos por cliente, facilitando análisis de uso y desempeño académico. Su integración con SCORM permite que el progreso registrado se clasifique por cliente, curso y usuario. La figura evidencia un backend orientado a escenarios reales de uso empresarial.

- generación de reportes básicos para seguimiento académico.

Este módulo permitió garantizar la trazabilidad del avance de los usuarios dentro del sistema.

Tercera actividad

Pruebas y validación del soporte SCORM

Para validar el correcto funcionamiento del módulo SCORM, se realizaron pruebas en el entorno de desarrollo configurado con **Docker**, utilizando **Cursor** como entorno de programación y herramientas como Postman y el reproductor SCORM del frontend.

Las pruebas incluyeron:

- **Ejecución de sesiones SCORM** desde el player en Vue 3, verificando la comunicación con los endpoints del backend y el envío de datos como estado, tiempo y puntaje.
- **Validación del almacenamiento en MySQL**, comprobando mediante consultas y registros que el progreso del usuario se guardara y actualizara correctamente dentro del contenedor de base de datos.
- **Pruebas funcionales y de respuesta** del backend usando Postman y registros del navegador, midiendo tiempos de respuesta y verificando el procesamiento de solicitudes concurrentes.
- **Revisión de logs de Docker**, identificando errores, advertencias y confirmando la correcta ejecución de los contenedores Laravel, Nginx y MySQL.
- **Documentación técnica del proceso**, registrando flujos de datos, endpoints utilizados y capturas de pruebas como evidencia del funcionamiento.

Resultado:

El soporte SCORM fue validado exitosamente, demostrando estabilidad, correcta comunicación entre frontend y backend, y almacenamiento confiable del progreso en la base de datos dentro del entorno Docker.

Conclusiones

El desarrollo del backend de la plataforma e-learning permitió construir una arquitectura sólida, modular y escalable basada en Laravel 11. La organización del código mediante controladores, servicios, repositorios y políticas contribuyó a un sistema limpio, ordenado y preparado para su crecimiento. Esta estructura técnica permitió cumplir con los requisitos iniciales del proyecto y sentó las bases para integrar funcionalidades más complejas a futuro.

La implementación de Docker fue un componente clave para asegurar la estabilidad del entorno de desarrollo. Gracias al uso de contenedores, el sistema pudo ejecutarse de manera consistente en cualquier equipo, evitando problemas de compatibilidad y facilitando las pruebas del backend, la base de datos y las APIs. Esto permitió simular un entorno real de producción y mejorar la calidad del desarrollo.

En cuanto a la seguridad, la integración de los sistemas de autenticación, roles y permisos mediante Laravel Sanctum y Spatie Permissions fortaleció la administración de accesos y garantizó una protección adecuada de los recursos. El panel administrativo construido con FilamentPHP permitió gestionar usuarios, cursos, módulos y actividades de una manera visual e intuitiva, cumpliendo con los estándares modernos de administración dentro de plataformas e-learning.

Un avance significativo del proyecto fue la integración inicial del estándar SCORM, que permitió establecer la comunicación entre el reproductor del frontend y el backend. Aunque la implementación completa del estándar requiere fases adicionales, se logró registrar el progreso académico de los usuarios, almacenando datos esenciales para la trazabilidad del aprendizaje. Esto constituye un paso importante hacia un LMS interoperable y alineado con estándares internacionales.

El uso de Scrum como metodología permitió organizar el desarrollo en sprints, priorizar tareas y asegurar una entrega continua de funcionalidades. Adaptar esta metodología a un trabajo individual facilitó mantener el control del proyecto y permitió avanzar de forma estructurada, minimizando errores y garantizando la claridad de los procesos.

En términos generales, el proyecto alcanzó aproximadamente un 65% de avance, cumpliendo con el primer objetivo y desarrollando una parte considerable de los demás. El sistema queda preparado para continuar su evolución sin necesidad de modificar la arquitectura existente. Esta práctica profesional permitió fortalecer las competencias técnicas, metodológicas y profesionales del desarrollador, consolidando la experiencia necesaria para crear soluciones backend orientadas al ámbito e-learning con altos estándares de calidad.

Recomendaciones

Completar la integración SCORM en su totalidad, profundizando en la compatibilidad con SCORM 2004, extendiendo los endpoints del tracking y agregando nuevos mecanismos de análisis del aprendizaje.

Implementar pruebas unitarias y pruebas end-to-end, especialmente para los módulos críticos como autenticación, cursos, módulos, actividades y SCORM, asegurando la estabilidad del sistema en ambientes de producción.

Optimizar el panel administrativo incorporando más filtros, acciones masivas y vistas personalizadas, lo que permitirá un mejor control para administradores y gestores educativos.

Agregar un sistema de logs avanzado, utilizando herramientas como Laravel Telescope o Monolog para registrar errores, accesos, tiempos de respuesta y datos del player SCORM.

Implementar un sistema de almacenamiento tipo bucket, como Amazon S3 o DigitalOcean Spaces, con el fin de optimizar el manejo de imágenes, documentos y paquetes SCORM en un entorno productivo.

Realizar pruebas de rendimiento y estrés sobre la API, especialmente en endpoints SCORM que recibirán múltiples interacciones por parte de los estudiantes durante sus actividades de aprendizaje.

Establecer un pipeline CI/CD, que permita realizar despliegues automáticos, pruebas automatizadas y control de versiones de manera profesional.

Desarrollar completamente el frontend en Vue 3, integrando los endpoints ya creados y permitiendo la navegación, visualización de cursos, reproducción SCORM y panel para estudiantes.

Fortalecer la documentación técnica del sistema, incluyendo diagramas de arquitectura actualizados, manuales de instalación, flujos de datos y guías para nuevos desarrolladores.

Considerar soporte progresivo para otros estándares e-learning, como xAPI o LTI, para ampliar la interoperabilidad de la plataforma y posicionarla como una solución más robusta en el mercado educativo

Referencias

- [1] F. Docs, 2025. [En línea]. Available: <https://filamentphp.com/docs>.
- [2] V. Docs, 2025. [En línea]. Available: <https://vuejs.org/guide/introduction.html>.
- [3] D. Estudio, «E-learning y desarrollo de apps innovadoras,» [En línea]. Available: <https://www.dominioestudio.com/dominio-estudio-elearning-y-desarrollo-de-apps-agiles-e-innovadoras-en-colombia-y-latinoamerica>.
- [4] Laravel, «Laravel Documentation,» 2025. [En línea]. Available: <https://laravel.com/docs/12.x>.
- [5] D. Inc, «Docker Documentation,» [En línea]. Available: <https://docs.docker.com>.
- [6] D. Inc, 2025. [En línea]. Available: <https://docs.docker.com/compose/.b>
- [7] O. M. Docs. [En línea]. Available: <https://dev.mysql.com/doc/>.
- [8] Atlassian, «Bitbucket,» [En línea]. Available: <https://bitbucket.org/product/>.
- [9] K. S. a. J. Sutherland, «The Scrum Guide,» 2020. [En línea]. Available: <https://scrumguides.org>.
- [10] Swagger. [En línea]. Available: <https://swagger.io/specification/>.
- [11] S. Docs. [En línea]. Available: <https://spatie.be/docs/laravel-permission/v6/introduction>.